

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра инженерной психологии и эргономики

Специальность «Программная инженерия»

Учебная дисциплина «Современные языки программирования»

ОТЧЕТ

по лабораторной работе №6
«Ведение журнала Python и обработка исключений»
Вариант 22

Подготовил:

Конон С.И.

Проверила:

Семерник О.Д.

Минск 2026

Задание:

Имплементировать калькулятор с логированием операций:

Обработка ошибок деления на ноль, неверного ввода, попытки логарифмирования отрицательного числа.

Логировать все действия, в том числе ошибочные, в файл calc.log.

Листинг кода:

```
import logging
import math
```

```
logging.basicConfig(
    level=logging.INFO,
    filename='calc.log',
    filemode='a',
    format='%(asctime)s - %(levelname)s - %(message)s',
    encoding='utf-8'
)
```

```
OPERATIONS = [
    "1. Сложение",
    "2. Вычитание",
    "3. Умножение",
    "4. Деление",
    "5. Возведение в степень",
    "6. Десятичный логарифм (только для первого числа)"
]
```

```
def calculate():
    print("Калькулятор")

    try:
        line = input("Введите два числа через пробел: ")
        a_raw, b_raw = line.split()
        a, b = float(a_raw), float(b_raw)
        logging.info(f"Введены числа: {a}, {b}")

        for op in OPERATIONS:
            print(op)

        choice = input("Выберите номер операции: ")
        logging.info(f"Выбрана операция: {choice}")

        result = None
        match choice:
            case "1":
```

```

        result = a + b
        logging.info(f"Сумма: {result}")
    case "2":
        result = a - b
        logging.info(f"Разность: {result}")
    case "3":
        result = a * b
        logging.info(f"Умножение: {result}")
    case "4":
        if b == 0:
            logging.error(f"ОШИБКА: Деление на ноль!")
            raise ZeroDivisionError("Деление на ноль!")
        result = a / b
        logging.info(f"Деление: {result}")
    case "5":
        result = a ** b
        logging.info(f"Возведение в степень: {result}")
    case "6":
        if a <= 0:
            logging.error(f"ОШИБКА: Логарифм отрицательного числа
или нуля!")
            raise ValueError("Логарифм отрицательного числа или
нуля!")
        result = math.log10(a)
    case _:
        print("Ошибка: Неверный выбор операции.")
        logging.warning(f"Некорректный выбор операции: {choice}")
        return

if result is not None:
    print(f"Результат: {result}")
    logging.info(f"Результат: {result}")

except ValueError as e:
    error_msg = f"Ошибка ввода или значения: {e}"
    raise ValueError(error_msg)
    logging.error(error_msg)
except ZeroDivisionError as e:
    error_msg = f"Математическая ошибка: {e}"
    raise ZeroDivisionError(error_msg)
    logging.error(error_msg)
except Exception as e:
    error_msg = f"Непредвиденная ошибка: {e}"
    raise Exception(error_msg)
    logging.exception("Critical Error")

```

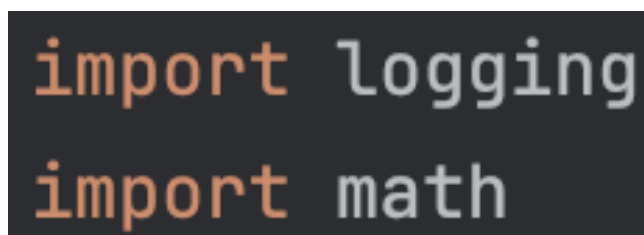
```
if __name__ == "__main__":  
    calculate()
```

Модуль logging в Python – это стандартный инструмент для записи сообщений о работе программы. В отличие от функции print(), он позволяет гибко управлять тем, какие сообщения выводить, куда их сохранять (консоль, файл, сервер) и в каком формате.

В модуле определены стандартные уровни важности сообщений:

- DEBUG: подробная информация для диагностики проблем;
- INFO: подтверждение того, что программа работает штатно;
- WARNING: индикация того, что произошло что-то неожиданное, но программа продолжает работать;
- ERROR: возникла серьезная проблема, часть функций не выполнена;
- CRITICAL: критическая ошибка, программа может завершить работу.

Для начала работы импортируем необходимые модули. Нам понадобятся logging для записи логов и math для вычисления логарифма.



```
import logging  
import math
```

Рисунок 1 – Импорт библиотек

Перед описанием логики калькулятора настроим конфигурацию логгера. Мы укажем файл для записи, уровень важности и формат сообщения (время – уровень – текст).

Ключевые параметры, которые используются в basicConfig:

- Level – минимальный порог сообщений.
- Format – структура строки лога.
- Datefmt – формат даты и времени.
- Filename – путь к файлу лога.
- Filemode – режим записи файла ('a' – дописывать в конец; 'w' – очищать файл при каждом запуске) .

```

logging.basicConfig(
    level=logging.INFO,
    filename='calc.log',
    filemode='a',
    format='%(asctime)s - %(levelname)s - %(message)s',
    encoding='utf-8'
)

```

Рисунок 2 – Настройка конфигурации логгера

Для удобства использования, создадим список OPERATIONS, где перечислим все операции:

```

OPERATIONS = [
    "1. Сложение",
    "2. Вычитание",
    "3. Умножение",
    "4. Деление",
    "5. Возведение в степень",
    "6. Десятичный логарифм (только для первого числа)"
]

```

Рисунок 3 – Список операций

Основная часть программы заключена в функцию calculate(). Программа запрашивает у пользователя данные, предлагает выбрать операцию и обрабатывает результат.

В данной программе реализована многоуровневая система защиты от ошибок:

- ValueError: Перехватывает ситуации, когда пользователь вводит текст вместо чисел или когда в функцию log10 передается отрицательное число.
- ZeroDivisionError: Защищает программу от критического завершения при попытке деления на ноль.
- Logging.exception: Автоматически записывает в файл calc.log не только текст ошибки, но и Traceback (путь ошибки в коде), что крайне полезно для отладки.

После нескольких запусков файл логов выглядит следующим образом:

```
2026-03-06 21:53:50,170 - INFO - Введены числа: 3.0, 6.0
2026-03-06 21:53:53,937 - INFO - Выбрана операция: 5
2026-03-06 21:53:53,937 - INFO - Возведение в степень: 729.0
2026-03-06 21:53:53,938 - INFO - Результат: 729.0
2026-03-07 08:40:18,187 - INFO - Введены числа: 2.0, 4.0
2026-03-07 08:40:23,885 - INFO - Выбрана операция: 5
2026-03-07 08:40:23,885 - INFO - Возведение в степень: 16.0
2026-03-07 08:40:23,885 - INFO - Результат: 16.0
```

Рисунок 4 – Файл с логами

Результат работы программы показан на рисунках 5-8.

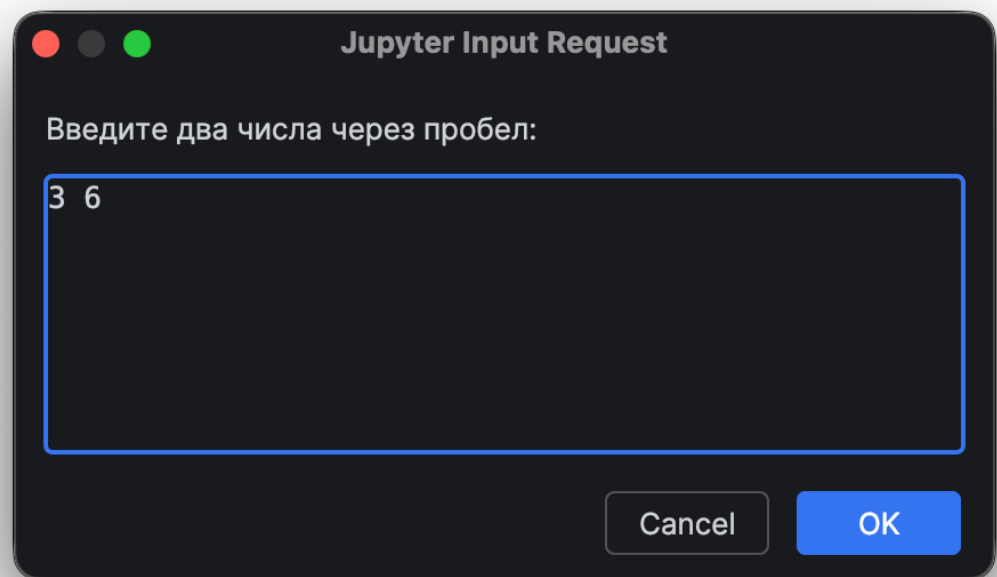


Рисунок 3 – Ввод двух чисел для вычисления

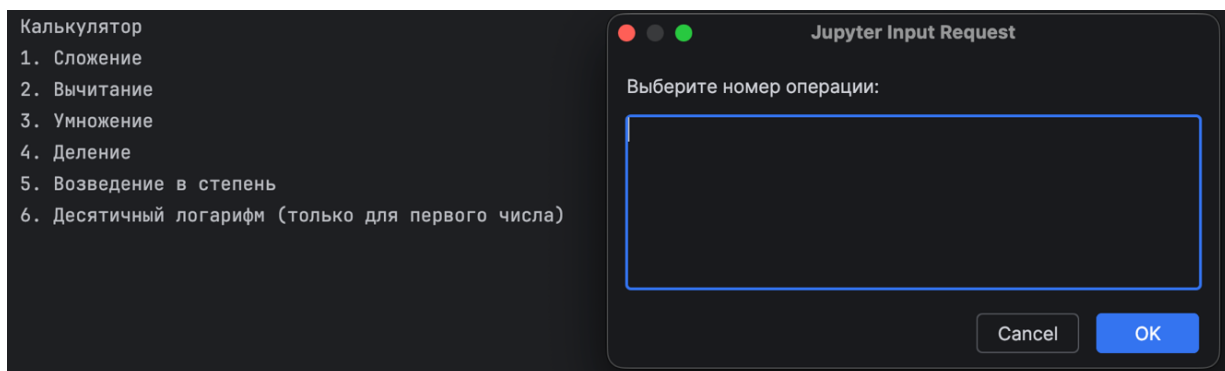


Рисунок 4 – Выбор операции

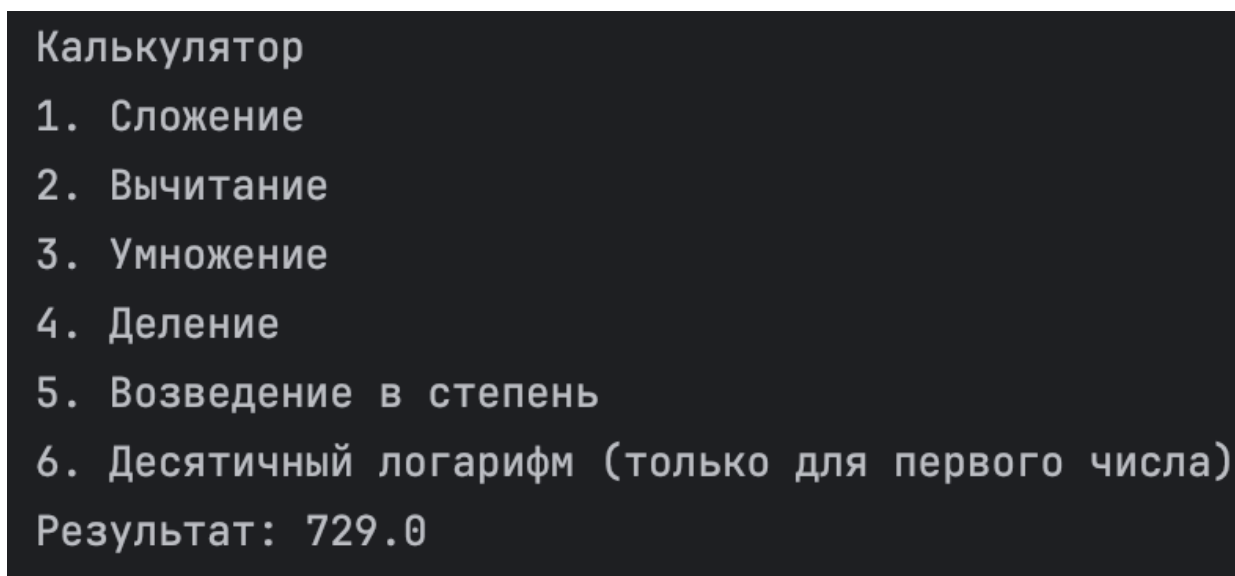


Рисунок 5 – Вывод результата

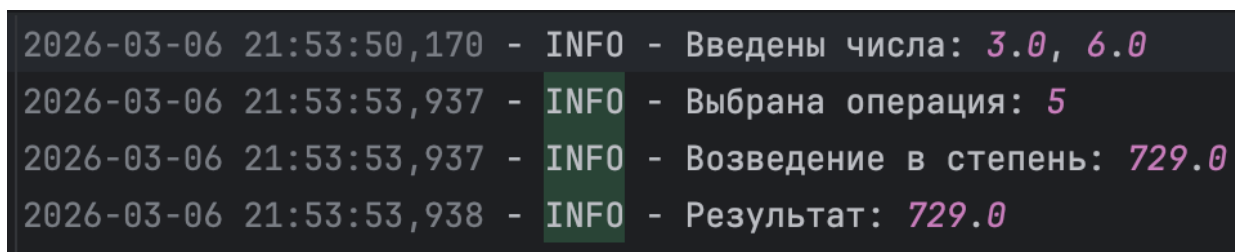


Рисунок 6 – Содержимое файла calc.log

Вывод:

В ходе выполнения данной лабораторной работы были практически освоены современные инструменты разработки на языке Python и методы создания отказоустойчивых приложений.

Основной акцент в работе был сделан на обеспечении отказоустойчивости (fault tolerance) программного продукта. Реализация комплексной системы обработки исключений с помощью конструкций try-

`except-else-finally` позволила эффективно нейтрализовать критические ошибки, такие как `ValueError` (некорректный тип данных при вводе), `ZeroDivisionError` (математическая неопределенность при делении на ноль) и `IndexError` (ошибки парсинга строки). Благодаря этому программа не прекращает работу аварийно, а предоставляет пользователю информативные сообщения, сохраняя свою работоспособность и стабильность даже при возникновении непредвиденных ситуаций.

Особое внимание было уделено внедрению системы системного логирования через встроенный модуль `logging`. В отличие от использования функции `print()`, которая выводит временные данные в консоль, логирование позволило:

- создать постоянный аудит событий в файле `calc.log`;
- разграничить события по уровням важности (`INFO`, `WARNING`, `ERROR`);
- сохранять трассировку стека (`traceback`) при возникновении критических ошибок, что в реальных условиях эксплуатации позволяет проводить «посмертный» анализ (`post-mortem debugging`) и восстанавливать последовательность действий пользователя, приведших к сбою.

Это делает программу более удобной для сопровождения, тестирования и последующего развития.

С точки зрения архитектуры кода, применение современного оператора ветвления `match-case` (введенного в Python 3.10) значительно повысило читаемость (`readability`) и поддерживаемость (`maintainability`) программы. Это позволило избежать нагромождения вложенных конструкций `if-elif-else` и сделало логику выбора операций более прозрачной, структурированной и легкой для расширения в будущем при добавлении новых функций или операций.

Дополнительно были закреплены навыки структурирования кода, разделения логики обработки данных и обработки ошибок, а также принципы написания более надежных программ, устойчивых к неправильным действиям пользователя. Использование журналирования и обработки исключений является важной частью разработки реальных программных систем, так как позволяет повысить их стабильность и упростить поиск ошибок в процессе эксплуатации.

В рамках лабораторной работы были закреплены навыки написания надежного, масштабируемого и структурированного программного обеспечения, соответствующего современным стандартам индустрии программирования на языке Python. Также был получен практический опыт работы с Jupyter Notebook как инструментом для ведения интерактивной документации, тестирования кода и фиксации этапов разработки программы.

Это способствует более эффективной организации процесса разработки и облегчает последующий анализ и доработку программного продукта.

Контрольные вопросы:

1. Исключение – это ошибка, возникающая во время выполнения программы, которая прерывает нормальный ход её выполнения. В Python исключения можно перехватывать и обрабатывать, чтобы программа не завершалась аварийно.

2. Встроенные типы исключений: `TypeError`, `ValueError`, `IndexError`, `KeyError`, `ZeroDivisionError`, `FileNotFoundError`, `ImportError`, `AttributeError` и другие.

3. `TypeError` возникает, когда операция выполняется с объектом неподходящего типа (например, сложение строки и числа). `ValueError` возникает, когда тип данных правильный, но значение недопустимо (например, преобразование строки "abc" в число).

4. Несколько исключений можно обработать в одном блоке `except`, перечислив их через запятую:

```
except (TypeError, ValueError):
```

5. Блок `finally` выполняется всегда – независимо от того, возникло исключение или нет. Обычно используется для освобождения ресурсов (например, закрытия файлов).

6. Блок `else` выполняется после блока `try`, если в нём не возникло исключений. Его используют для кода, который должен выполняться только при успешном выполнении `try`.

7. `raise` без аргумента повторно выбрасывает текущее исключение, которое было перехвачено в блоке `except`.

8. Собственное исключение создаётся через наследование от класса `Exception`:

```
class MyError(Exception):  
    pass
```

9. Родительским классом пользовательского исключения обычно является `Exception`, так как это базовый класс для большинства ошибок в Python.

10. `Traceback` – это подробная информация об ошибке, которая показывает последовательность вызовов функций и строк кода, где произошла ошибка.

11. Модуль `logging` используется для записи сообщений о работе программы (ошибки, предупреждения, информация). Это помогает отслеживать и анализировать работу приложения.

12. `logging` отличается от `print()` тем, что поддерживает уровни сообщений, может записывать информацию в файлы и гибко настраивается. `print()` просто выводит текст на экран.

13. Основные уровни логирования: `DEBUG` (подробная отладочная информация), `INFO` (обычные сообщения), `WARNING` (предупреждения), `ERROR` (ошибки), `CRITICAL` (критические ошибки).

14. Для записи логов в файл используется настройка:

```
logging.basicConfig(filename='app.log', level=logging.INFO)
```

15. Форматтер задаёт формат сообщений в журнале (например, время, уровень ошибки, текст сообщения). Это делает логи более удобными для анализа.

16. Логгер – это объект, который используется для записи сообщений. Его можно создать так:

```
logger = logging.getLogger(__name__)
```

17. Уровень логирования по умолчанию можно изменить с помощью:
`logging.basicConfig(level=logging.INFO)`

18. `logging.exception()` используется внутри блока `except` и записывает сообщение об ошибке вместе с полной трассировкой (`traceback`).

19. Исключения используются для обработки ошибок во время выполнения программы, а логирование — для записи информации о работе программы и её состоянии.

20. При работе с несколькими модулями обычно создают отдельный логгер в каждом модуле с помощью `logging.getLogger(__name__)`, что позволяет централизованно управлять журналом событий.